

User Mode Linux Redesign

Architecture, Implementation State, and Validation Evidence

Reporting cutoff:

`umlctl-deploy @ 485eff5c96be`

May 14, 2026

Goal and State

Reader Contract

Assumed reader

- knows Linux review culture
- may know classic UML
- does not know this diary
- does not know our phase names

What this deck must do

- state the value proposition
- separate inherited work from branch work
- explain `kvm-v2` without VMM confusion
- show evidence and remaining review risk

UML can become a profile-driven kernel research substrate again.

The redesign turns one source tree into fast production, observable research, fuzzing, sandbox, library, and time-travel artifacts.

backend abstraction

static-key hot paths

profile composition

sustained validation

Original promise

- Linux kernel as a userspace process
- rootless development and debugging
- isolated Linux instances
- no machine emulation requirement

Modern blockers

- userspace trap cost
- SMP and state ownership
- fuzzing throughput
- stale profile and tooling story

RESPONSE

backend abstraction + runtime gates + profile matrix + kvm-v2

Build from one tree

- near-native prod-fast
- sanitizer-heavy research
- KCOV/snapshot fuzz
- minimized sandbox
- direct-call library
- deterministic time-travel

Do not build

- a Linux reimplementation
- a QEMU/Firecracker replacement
- a container runtime
- a from-scratch hypervisor

Why Not Existing Tools?

Option	What it is good at	Why UML still matters
QEMU/KVM	General virtual machines, device models, booting real images.	UML is a kernel built as a host process; it is cheaper to instrument and easier to bind into source-level kernel tests.
Firecracker	Minimal production microVMs.	It optimizes a different surface: cloud isolation and boot footprint, not sanitizer-heavy kernel research inside <code>arch/um</code> .
gVisor	Userspace kernel sandboxing.	It proves useful backend patterns, but it is not the Linux kernel under test.
LKL	Direct-call Linux library harnesses.	It is fast, but loses the normal user/kernel ring split that matters for syscall and exploit research.
Containers	Process isolation and deployment.	They share the host kernel; they cannot test another Linux kernel as the target.

Current State at May 14, 2026

Cutoff	485eff5c96be on umlctl-deploy
Functional headline	kvm-v2 has working guest entry, syscall dispatch, exceptions, signals, SMP state repair, and post-gadget performance
Parity	cpython-parity 21/21 under UP and SMP
Substrate gate	PASS=25/FAIL=3/XFAIL=3, matching seccomp
2h soak	kvm-v2 200/200 PASS; aggregate 397/400 PASS
Benchmark	Faster than seccomp on all eight hosts and all three tiers

DONE

backend/gates/profile foundation

IN PROGRESS

sustained validation

BLOCKED

large upstream series until validation

Original Versus This Branch

Category	Reading
Inherited UML and Linux	Linux-as-a-host-process, legacy ptrace, generic kernel subsystems, existing KVM APIs, and the broad idea of time travel are foundations, not branch-local inventions.
Upstream prerequisites	Seccomp-mode UML and upstream SMP are enabling context that the redesign builds on.
Redesigned here	Backend ops, static-key gates, profile matrix, kvm-v2 state ownership, validation ladder, and upstream sequencing.
Implemented here	arch/um/backend/kvm-v2/, profile docs and configs, snapshot/forkserver v1, umlctl, launcher, gates, soak, benchmark, and parity selftests.
Planned or paused	KVM-aware snapshot readiness, full record/replay, time-machine workflow, syzkaller vm/uml, LTP, and upstream arch/um series packaging.
DIFF CHECK	origin/master...HEAD over UML paths: 688 files, 154,501 insertions, 823 deletions.

21/21

cpython parity
UP and SMP

200/200

kvm-v2 two-hour
validation pilot

193–908x

micro speedup
over seccomp

7 series

staged upstream
submission queue

Eight Answers: Why This Work Exists

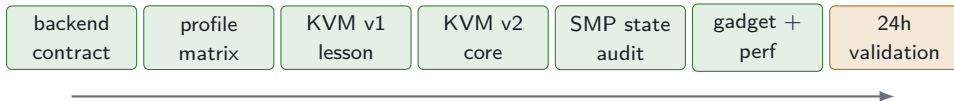
#	Question	Short answer
1	Revived value proposition	One upstream Linux tree can produce fast, research, fuzz, sandbox, library, and deterministic profiles.
2	Why not existing tools	They solve adjacent problems; none gives Linux-as-a-process with this instrumentation and syscall fidelity mix.
3	Shared architecture	Backend ops, static-key gates, and profile defconfigs keep variation explicit.
4	Landed vs planned	Core architecture, kvm-v2, tooling, and gates exist; 24h validation, LTP, syzkaller, and replay remain.

Eight Answers: What Reviewers Should Test

#	Question	Short answer
5	SMP and state	v2 now has an explicit state-home contract and audit trail for per-task, per-vCPU, and per-mm state.
6	Speed beyond micro	Python and stress tiers win on every benchmark host, not only getpid.
7	Upstream bar	Finish sustained validation and split small subsystem series before the large arch/um series.
8	Reviewer challenges	Challenge the data model, validation breadth, upstream slicing, and replay/snapshot claims.

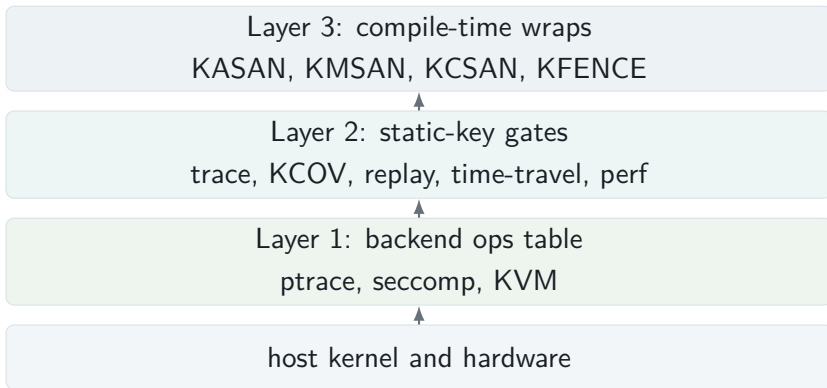
Architecture

Milestone Map, Plain English



- Internal phase labels exist in the diary, but reviewers should read the work as milestones.
- The current frontier is not a new design idea; it is sustained workload breadth before upstreaming.

Three-Layer Architecture



Layer 1: Backend Contract

Contract categories

- lifecycle and trap
- memory regions
- scheduling and IPI
- time
- debug/register access

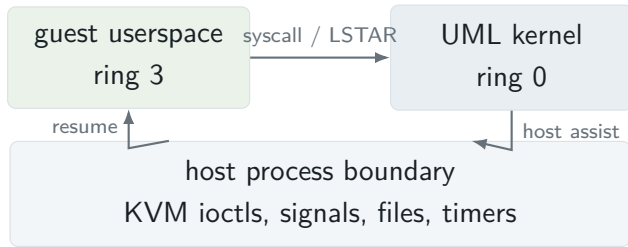
Design point

- every backend implements the table
- single-backend builds can inline calls
- dynamic builds select at boot
- capability flags replace global backend booleans

Profiles Are Product Points

Profile	Backend	Hooks	Role
prod-fast	KVM > seccomp	none	near-native production
prod-with-hooks	KVM > seccomp	compiled, off	production with emergency observability
research	seccomp	trace, kprobes	sanitizer/debug profile
fuzz	seccomp	KCOV, snapshot	syzkaller/harness fuzzing
fuzz-deep	seccomp	KCOV, replay	race and replay investigations
sandbox	seccomp-only	none compiled	minimized TCB
library	direct	n/a	LKL-style harnesses
time-travel	seccomp	deterministic time	replayable execution

KVM v2 Model



This is a trap mechanism for UML, not a general-purpose VMM.

What KVM v1 Taught Us

v1 design pressure

- treat KVM as a faster trap loop
- share too much vCPU/run state
- repair state after exits
- maintain shadow page-table state

lesson

- SMP turns repair workarounds into races
- `kvm_run` must have one active owner
- host signals need KVM-visible masking
- every state item needs an authoritative home

DESIGN SHIFT

from “fix the next trap” to an explicit state-ownership contract

Changed Data Model

State class	Authoritative home	Contract in v2
kvm_run and vCPU registers	per-host-CPU vCPU	Pick while migration is disabled; no other CPU owns that run page during dispatch.
Guest GPRs and syscall ABI	task registers plus sync-regs mmap	Marshal into KVM before entry and back to UML after exit at explicit boundaries.
FPU/XSAVE	per-task snapshots plus vCPU dirty epoch	Install on first use, capture when ownership changes, avoid cross-task XMM leakage.
IST and exception frames	per-task frame snapshots over per-vCPU pages	Restore before dispatch; snapshot interrupted stubs instead of reading another task's frame.
TLB state	per-mm generation plus per- vCPU last-seen state	Memory changes become generation changes; vCPUs refresh or get kicked.
Host signal delivery	KVM_SET_SIGNAL_MASK	SIGALRM cannot arbitrarily break the KVM window; a dedicated kick signal can.

The Hard Part Was State Ownership

Audit inputs

- 149 state items
- 50+ operations
- ownership matrix
- suspect register audits
- runtime state trace

Bug classes closed

- migration while using per-CPU vCPU state
- EINTR mid-stub register recovery
- cross-task FPU/XMM leakage
- worker fd leak and shutdown hang
- AVX/XSAVE #UD drift

Validation

Validation Ladder

tooling builds – launcher and umlctl

kernel builds – selected profile compiles and links

internal contracts – KUnit, marshal, byte-shape checks

substrate parity – PASS=25/FAIL=3/XFAIL=3, matching seccomp

process/syscall classes – fork, exec, mmap, signals

performance gates – prevent fast-path regression

real workload boot – Python and stress workloads

sustained validation – 24h soak plus Tier 2/3 and LTP curation

Sustained Validation: First Pilot

Tuple	n	pass	fail	rate
kvm-v2 aggregate	200	200	0	100.00%
seccomp aggregate	200	197	3	98.50%
total	400	397	3	99.25%

- This is a pilot for the 24-hour validation bar, not the final upstream gate.
- 35 minutes wall before threshold-trip clean stop.
- Three failures were ioccheck/seccomp timeout shape, not data corruption.
- kvm-v2 passed every iteration across memcheck, ioccheck, stress-ng, and tier1-pylibs.

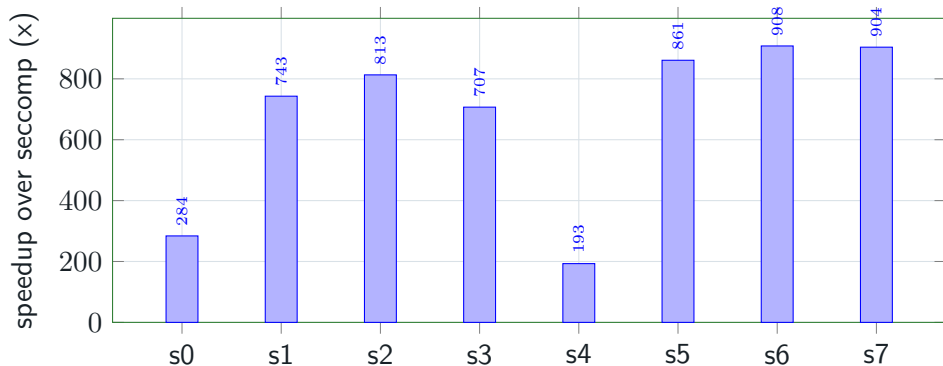
Performance

How To Read the Benchmarks

Tier	What it measures	Why it matters
Micro	RDTSC-bracketed getpid-style loop.	Isolates the trap mechanism and LSTAR gadget cost.
Native context	Profile cost model relative to normal Linux getpid.	Prevents a speedup-over-seccomp chart from hiding absolute scale.
Python	Startup and mixed-syscall user-land workload.	Checks whether the micro win survives real imports and dynamic linking.
Stress	SMP memory and page-fault-heavy workload.	Checks whether performance survives correctness pressure.

Charts use same-host speedup over seccomp because raw cycles are not portable across CPU generations. The native baseline table gives the human-scale reference point.

Micro Benchmark: The Gadget Changed the Cost Class



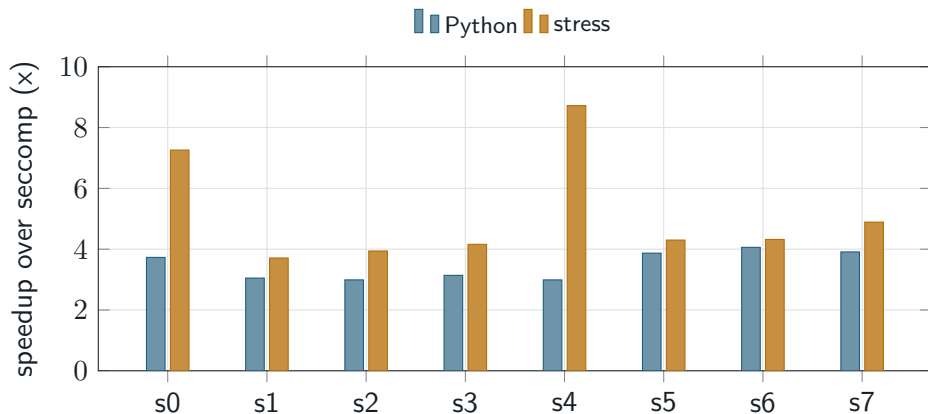
This chart answers only the narrow mechanism question: a gadget-covered trivial syscall no longer pays the seccomp signal/stub round trip.

Native Baseline Context

Path	Relative to normal	Reading
Normal Linux getpid	1.0x, 50 ns	Baseline used by the profile cost model.
kvm-v2 prod-fast	1.6x, 80 ns	Same order as native for gadget-covered calls.
seccomp profiles	10–24x, 505–1200 ns	Expected for research, fuzz, sandbox, and time-travel.
ptrace fallback	40x, 2,000 ns	Compatibility path, not the speed target.
direct library	0.12x, 6 ns	Fast because it removes the ring split.

The cross-host benchmark still reports same-host speedup over seccomp; this table provides user-facing native-system context.

Python and Stress Tiers



Python checks userland startup and dynamic-linking behavior; stress checks SMP/page-fault pressure. Both are same-host speedups over seccomp.

Roadmap

Snapshot, Replay, Time Travel

Surface	State	Reading
Snapshot/forkserver v1	DONE	um_snapshot_ready(), fds 198/199, state_version, launcher plumbing, and snapshot-smoke are present.
v1 ceiling	LIMIT	status is hard-coded zero; no on-disk snapshot; short non-blocking payloads only; no SMP.
KVM-aware readiness	PLANNED	Issue #168 is unblocked but sits behind sustained validation.
Time-travel profile	IN PROGRESS	UP/seccomp profile exists; runtime hook gate exists; slow path is still a counter stub.
Record/replay/time machine	PLANNED	Issue #169 covers the real event log, replay consumer, and workflow.

What Remains

Validation

- formal 24h sustained run
- Tier 2/Tier 3 templates
- LTP curation
- wider CI matrix

Resumed work

- KVM-aware snapshot readiness
- record/replay extensions
- syzkaller vm/uml
- optional AVX-512 follow-up

Upstream Path

#	Series	Subsystem	State
1	bpf-hygiene-v1	BPF	ready
2	kmsan-arch-callback-rfc	mm/kmsan	ready
3	ftrace-notrace-generic-v1	tracing	prepared
4	backend ops abstraction RFC	arch/um	to write
5	static-key hot paths	arch/um	to write
6	per-profile C-series	arch/um + subsystems	blocked
7	KVM backend	arch/um + KVM	blocked by sustained validation

What Reviewers Should Challenge

Architecture

- Is the backend contract minimal?
- Are state homes explicit enough?
- Does `kvm-v2` leak VMM assumptions into UML core?
- Are profile defaults defensible upstream?

Evidence

- Is the validation ladder broad enough?
- Are seccomp comparisons the right control?
- Which LTP failures should be KEEP vs SKIP?
- Are snapshot/replay claims scoped tightly?

The redesign is now an implementation with a validation frontier.

- The architectural shape is coherent: backends, gates, profiles.
- `kvm-v2` has crossed the hardest feasibility line: real workloads, SMP, and speed.
- The next credible milestone is not another design memo; it is sustained validation and an upstreamable series plan.

Backup: Detailed Speedups

Host	CPU	micro	Python	stress
s0	i9-12900K	284x	3.73x	7.26x
s1	Xeon E3-1225 v6	743x	3.05x	3.71x
s2	Xeon E3-1225 v5	813x	2.99x	3.94x
s3	Xeon W-2123	707x	3.14x	4.16x
s4	i5-12600K	193x	2.99x	8.72x
s5	Ryzen 7 7840HS	861x	3.87x	4.30x
s6	Ryzen 7 7840HS	908x	4.06x	4.32x
s7	Ryzen 7 7840HS	904x	3.91x	4.89x

Backup: Source Map

- `00-vision.md`: goals, non-goals, stakeholders.
- `STATUS.md`: live state, phase ledger, blockers.
- `01-architecture/`: three layers, invariants, data flow.
- `02-workstreams/D-kvm-backend/`: KVM design, v1-to-v2 restart memos, state audit, benchmarks, sustained validation.
- `03-architecture-review-2026-04-27/`: root-cause synthesis for the v1 contract mismatch.
- `03-profiles/`: profile matrix and cost model.
- `tools/testing/selftests/um/`: gates, benchmarks, soak, parity.
- `tools/uml/uml-launcher/`: launcher and `umlctl`.